

ZMX & C-TRADE PLATFORMS



WHATSAPP NLP CHATBOT MODULE

Scope of Work — Innovation Hub Challenge No 2

April 2026

1. Overview

The ZMX and C-Trade platforms are seeking to expand access to financial markets through conversational technology. The challenge is building a Natural Language Processing (NLP) module that allows investors to interact with both platforms entirely through voice prompts delivered over WhatsApp powered by the Twilio API.

Users will be able to speak (or type) natural language commands such as "Buy 100 shares of Econet" or "Show my last five transactions" and have the system understand the intent, execute the action against the live APIs/MSSQL DBs, and respond with a clear confirmation all within a WhatsApp conversation.

1.1 Background

ZMX (Zimbabwe Mercantile Exchange) and C-Trade are regulated financial trading platforms operating in Zimbabwe. A large segment of the target investor base is mobile-first and may find traditional trading interfaces complex. A WhatsApp-based chatbot with voice understanding significantly lowers the barrier to entry and brings stock market participation to everyday users.

1.2 Target Platforms

Platform	Description	Use Case
ZMX	Zimbabwe Mercantile Exchange	Commodity and equity trading via conversational commands
C-Trade	Investor-facing trading portal	Stock purchase, portfolio view and transaction history via WhatsApp

2. Scope of Work

Participants are required to design and build a working NLP chatbot module with the following capabilities. The solution must function end-to-end: from receiving a voice or text prompt on WhatsApp, through understanding the user's intent, to executing the correct database operation and returning a plain-language response.

2.1 Core Features Required

#	Feature	Description	Expected Output
1	Buy Stocks	User says or types a buy instruction (e.g. "Buy 50 Econet shares"). The NLP layer extracts the stock ticker, quantity and action.	Order inserted into the MSSQL orders table; WhatsApp confirmation sent to user.
2	Sell Stocks	User issues a sell instruction (e.g. "Sell my 30 Delta shares"). System validates current holdings before executing.	Sell order recorded in MSSQL; confirmation with updated balance returned.
3	View Portfolio	User asks to see their current holdings (e.g. "What stocks do I own?" or "Show my portfolio").	Live portfolio data fetched from MSSQL and returned as a formatted WhatsApp message.
4	Transaction History	User requests recent activity (e.g. "Show my last 10 transactions" or "What did I buy this	Transaction records retrieved from MSSQL and presented in a

		week?").	readable list.
5	Stock Price Check	User asks for the current price of a stock (e.g. "What is the price of CBZ?").	Latest price fetched from the database or market data source and returned.
6	Account Balance	User asks for their available cash balance (e.g. "How much money do I have?").	Account balance record retrieved from MSSQL and returned.

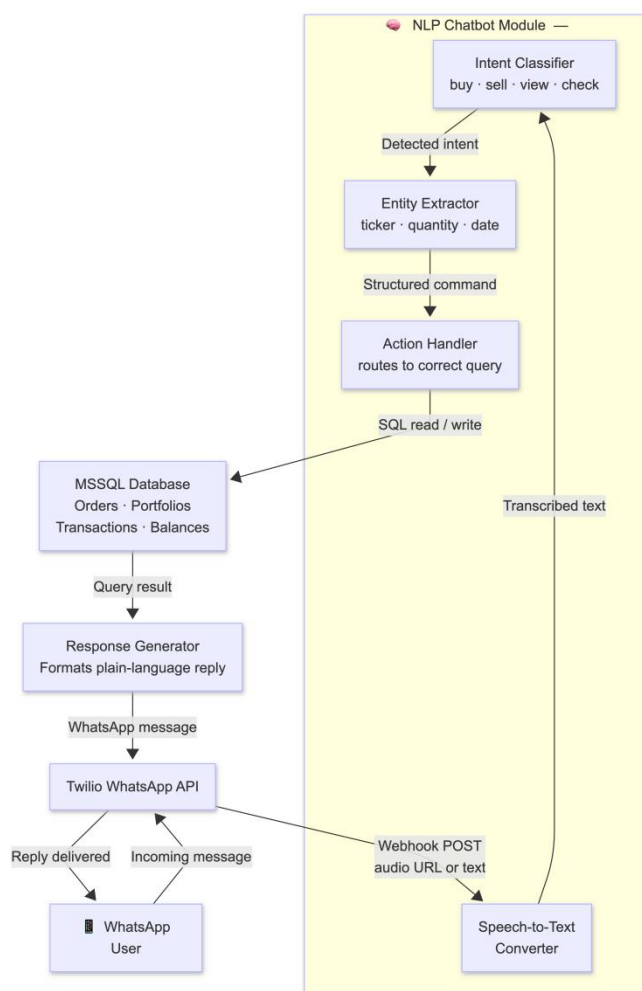
2.2 What Is Out of Scope

Participants are not required to build the following (these are provided or pre-exist):

- The core ZMX or C-Trade trading engine
- Payment processing or fund transfers
- KYC / identity verification flows
- The production MSSQL database schema (a test schema will be provided)

3. High-Level Architecture

The diagram below shows how the components fit together. The Twilio WhatsApp integration, MSSQL database and a sample dataset will be provided.



4. Technical Requirements

4.1 Mandatory Components

Component	Requirement	Suggested Approach
Speech-to-Text	Convert WhatsApp voice notes to text for NLP processing	OpenAI Whisper, Google Speech-to-Text, or Azure Speech
Intent Detection	Identify user intent from the transcribed or typed message (buy, sell, view, check)	Custom classifier, spaCy, Rasa NLU, or GPT function-calling
Entity Extraction	Extract stock ticker, quantity, date range and other parameters from the message	Named Entity Recognition (NER) model or regex-based fallback
Action Handler	Map detected intent + entities to the correct database operation	Python / Node.js service layer
MSSQL Integration	Read and write to the provided MSSQL test database	pyodbc, mssql npm package, or any ODBC-compatible driver
Twilio Integration	Receive incoming WhatsApp messages and send replies via Twilio API	Twilio Python / Node.js SDK (credentials provided)
Response Generator	Format database results into a clear, friendly plain-language WhatsApp reply	Template-based or LLM-assisted formatting

4.2 Database Tables (Provided)

Participants will be given access to a test MSSQL database with the following tables pre-populated with sample data:

4.3 Language & Framework

Participants are free to use any programming language or framework they are comfortable with. The following are recommended but not mandatory:

- Python (FastAPI or Flask) — recommended for NLP tasks
- Node.js (Express) — recommended for Twilio webhook handling
- Any cloud-hosted NLP/AI API is permitted
- The solution must be deployable and demonstrable within the challenge timeframe

5. Deliverables

Each participant must submit the following at the end of the challenge:

#	Deliverable	Description
1	Working Prototype	A live, demonstrable WhatsApp chatbot that can handle all six core features listed in Section 2.1. Must be accessible via a real WhatsApp number connected to Twilio.
2	Source Code	Full source code submitted to the provided Git repository. Code must be clean, commented and include a README with setup instructions.
3	Database Script	Any additional SQL scripts used to extend or seed the test database,

		submitted alongside the source code.
4	Presentation Slides	A brief slide deck (max 10 slides) covering: problem, approach, architecture, demo highlights, and known limitations.

6. Judging Criteria

Submissions will be evaluated by the Innovation Hub panel on the following criteria:

Criterion	Weight	What Judges Are Looking For
NLP Accuracy	30%	How well does the system understand varied phrasings, abbreviations and partial commands?
Feature Completeness	25%	How many of the six core features are working end-to-end?
Database Integration	20%	Are MSSQL reads and writes correct, reliable and properly handled?
User Experience	15%	Are WhatsApp responses clear, concise and easy to understand for a non-technical investor?
Code Quality	10%	Is the code well-structured, documented and reproducible from the README?